```
public IActionResult OnPost()
{
    if(!string.IsNullOrWhiteSpace(Input.CountryName) &&
       !string.IsNullOrWhiteSpace(Input.CountryCode) &&
       Input.CountryName.ToLower().First() !=
       Input.CountryCode.ToLower().First())                          ❶
    {
        ModelState.AddModelError("Input.CountryName",
        ➥ "The first letters of the name and code must match");     ❷
    }
    if (ModelState.IsValid)
    {
        CountryCode = Input.CountryCode;
        CountryName = Input.CountryName;
        return RedirectToPage("/CountryManager/Success");
    }
    return Page();
}
```

❶ The validation test is applied.

❷ If the test returns false, an error is logged with ModelState as a property-level entry.

The version of the `AddModelError` method you are most likely to use takes two strings: the name of the property that the error applies to and the error message you want to show the user. If the name of the property matches one passed in to a validation tag helper, that tag helper will display the error message.

Since this error message applies to two form fields, you may decide you don't want it to appear against one without the other. To prevent this issue, you can add another error message relating to the other property:

```
ModelState.AddModelError("Input.CountryName",
    "The first letter must match the first letter of the ISO 3166-1 code");
ModelState.AddModelError("Input.CountryCode",
    "The first letter must match the first letter of the country name");
```

Alternatively, you can pass an empty string to the property name, making this a model-level entry, which will be displayed by the validation summary tag helper whether it is set to `ModelOnly` or `All`: